

OPERATING SYSTEMS LABORATORY

PROGRAM 3: Priority and Round Robin Scheduling

Action-Marked Smartboard Guide | Python | Step-by-Step

CLASS OBJECTIVE Implement and compare two CPU scheduling algorithms: non-preemptive Priority Scheduling and preemptive Round Robin Scheduling. Students will understand priority, time quantum, remaining time, ready queue, execution slices, and scheduling order.		
Algorithm	Main Rule	Type
Priority Scheduling	Choose the ready process with the best priority. Smaller number = higher priority.	Non-preemptive
Round Robin	Each ready process gets CPU for at most the time quantum.	Preemptive
STANDARD DATA USED IN THIS GUIDE P1(AT=0, BT=7, Priority=2), P2(AT=2, BT=4, Priority=1), P3(AT=4, BT=1, Priority=3), P4(AT=5, BT=4, Priority=2). Round Robin quantum = 2. Smaller priority number means higher priority.		
ACTION MARKING USED ON EVERY PAGE GREEN = type in terminal and press Enter once. BLUE = type inside program3.py. GRAY = do not type; it is expected output, diagram, or explanation. ORANGE = optional.		

1. Start Program 3 Without Losing Programs 1 and 2

YOUR PREVIOUS PROGRAMS ARE SAFE

Program 1 is in ~/OS-First-Program and Program 2 is in ~/OS-Second-Program. Today we create a NEW folder ~/OS-Third-Program. Nothing is deleted.

STEP 1 | Open Ubuntu and Go to Home Directory

TYPE IN UBUNTU / VS CODE TERMINAL - PRESS ENTER ONCE

```
cd ~
```

STEP 2 | See Your Existing Program Folders

TYPE IN UBUNTU / VS CODE TERMINAL - PRESS ENTER ONCE

```
ls
```

DO NOT TYPE - EXAMPLE ONLY; YOUR LIST MAY CONTAIN MORE FOLDERS

```
OS-First-Program
```

```
OS-Second-Program
```

```
...
```

STEP 3 | Create the New Folder for Program 3

TYPE IN UBUNTU / VS CODE TERMINAL - PRESS ENTER ONCE

```
mkdir -p ~/OS-Third-Program
```

STEP 4 | Enter Program 3 Folder and Verify

TYPE IN UBUNTU / VS CODE TERMINAL - PRESS ENTER ONCE

```
cd ~/OS-Third-Program
```

```
pwd
```

DO NOT TYPE - EXPECTED OUTPUT / REFERENCE ONLY

```
/home/your_username/OS-Third-Program
```

2. Open VS Code and Create program3.py

STEP 5 | Open Current Folder in VS Code

TYPE IN TERMINAL - PRESS ENTER ONCE

```
code .
```

code = open Visual Studio Code

. = open the current folder

Checkpoint: VS Code should show OS-Third-Program.

STEP 6 | Create the Python File

In VS Code Explorer, click New File and type:

TYPE AS THE FILE NAME - THEN PRESS ENTER

```
program3.py
```

After the file opens, Python code will be typed inside this file.

IMPORTANT

Do not type Python statements at the Ubuntu \$ prompt. Type them inside program3.py. In the editor, press Enter to move to the next code line. After all four code parts are complete, press Ctrl + S to save.

BEFORE CODING: 6 TERMS

AT: Arrival Time - when a process enters the ready system.

BT: Burst Time - total CPU time required.

Priority: Declared importance. Here smaller number = higher priority.

Quantum: Maximum CPU time in one Round Robin turn.

Remaining Time: CPU time still needed by a process.

Ready Queue: FIFO line of processes waiting for CPU.

3. Type Program 3 - Part 1: Import and Input Data

TYPE IN program3.py - PART 1 OF 4 - USE ENTER FOR NEW LINES; DO NOT RUN YET

```
from collections import deque

processes = [
    {"pid": "P1", "arrival": 0, "burst": 7, "priority": 2},
    {"pid": "P2", "arrival": 2, "burst": 4, "priority": 1},
    {"pid": "P3", "arrival": 4, "burst": 1, "priority": 3},
    {"pid": "P4", "arrival": 5, "burst": 4, "priority": 2},
]
```

Code / Term	Meaning
from ... import ...	Import only a required tool from a Python module.
collections	Python module containing useful container data structures.
deque	Double-ended queue; used here as the Round Robin ready queue.
processes = [...]	Create a list named processes.
{...}	A dictionary storing one process record.
pid	Process label: P1, P2, P3, P4.
arrival	Arrival Time (AT).
burst	Burst Time (BT).
priority	Priority value; smaller means higher in this practical.

DO NOT RUN YET
Part 1 only stores the input. Continue typing Part 2 below the closing] in the same program3.py file.

4. Type Program 3 - Part 2: Priority Scheduling Function

TYPE BELOW PART 1 IN `program3.py` - PART 2A OF 4 - USE ENTER FOR NEW LINES; DO NOT RUN YET

```
def priority_scheduling(processes):
    current_time = 0
    completed = set()
    result = []

    while len(completed) < len(processes):
        ready = [
            p for p in processes
            if p["arrival"] <= current_time and p["pid"] not in completed
        ]

        if not ready:
            next_time = min(
                p["arrival"] for p in processes
                if p["pid"] not in completed
            )
            result.append(("IDLE", current_time, next_time))
            current_time = next_time
            continue
```

INDENTATION RULE

All lines inside `def` are indented. Lines inside `while`, `if`, and list brackets are indented further. Copy the spacing exactly.

5. Continue Priority Scheduling Function

CONTINUE DIRECTLY BELOW PART 2A - PART 2B OF 4 - USE ENTER FOR NEW LINES; DO NOT RUN YET

```
p = min(
    ready,
    key=lambda x: (x["priority"], x["arrival"], x["pid"])
)

start = current_time
end = start + p["burst"]
result.append((p["pid"], start, end))
current_time = end
completed.add(p["pid"])

return result
```

Term	Simple Meaning
def	Define/create a function.
current_time	The current CPU clock time used by the simulation.
set()	Creates an empty set; completed PIDs are stored here.
while	Repeat while a condition remains true.
len(...)	Number of items.
ready	Processes that have arrived and are not completed.
min(ready, key=...)	Choose the best item according to a rule.
lambda	A short inline rule used here for sorting/selection.
priority, arrival, pid	Tie-break order: priority first, then arrival, then PID.
append	Add an execution interval to result.
completed.add	Mark the process as finished.
return	Send the final result back from the function.

6. Understand Non-Preemptive Priority Scheduling

PRIORITY CONVENTION			
Smaller number = higher priority. Priority 1 is higher than Priority 2, and Priority 2 is higher than Priority 3.			
Time	Ready Processes	Selection	Reason
0	P1	P1	Only P1 has arrived.
7	P2(pr=1), P3(pr=3), P4(pr=2)	P2	Priority 1 is highest.
11	P3(pr=3), P4(pr=2)	P4	Priority 2 is higher than 3.
15	P3	P3	Only P3 remains.
DO NOT TYPE - PRIORITY GANTT / DRY-RUN REFERENCE			
P1 [0-7] -> P2 [7-11] -> P4 [11-15] -> P3 [15-16]			
WHY DOES P2 NOT INTERRUPT P1 AT TIME 2?			
Because this Priority Scheduling implementation is NON-PREEMPTIVE. Once P1 starts, it continues until its burst finishes at time 7.			

7. Type Program 3 - Part 3: Round Robin Function

TYPE BELOW PRIORITY FUNCTION - PART 3A OF 4 - USE ENTER FOR NEW LINES; DO NOT RUN YET

```
def round_robin(processes, quantum):
    if quantum <= 0:
        print("Quantum must be greater than 0")
        return []

    process_list = sorted(
        processes,
        key=lambda x: (x["arrival"], x["pid"])
    )

    remaining = {p["pid"]: p["burst"] for p in process_list}
    queue = deque()
    result = []
    current_time = 0
    i = 0
```

Term	Meaning
quantum	Maximum CPU time allowed in one Round Robin turn.
quantum <= 0	Invalid because quantum must be positive.
sorted(...)	Creates processes in arrival/PID order.
remaining	Dictionary storing CPU time still required.
deque()	Creates the FIFO ready queue.
i	Index pointing to the next process not yet inserted into queue.

8. Continue Round Robin Function

CONTINUE PART 3 DIRECTLY - PART 3B OF 4 - USE ENTER FOR NEW LINES; DO NOT RUN YET

```
while i < len(process_list) or queue:
    if not queue:
        if current_time < process_list[i]["arrival"]:
            result.append((
                "IDLE",
                current_time,
                process_list[i]["arrival"]
            ))
            current_time = process_list[i]["arrival"]

        while i < len(process_list) and process_list[i]["arrival"] <= current_time:
            queue.append(process_list[i])
            i += 1
```

Expression	Meaning
while ... or queue	Continue while an unqueued process exists OR the ready queue is not empty.
if not queue	The ready queue is empty.
IDLE	CPU has no ready process; move time to the next arrival.
queue.append(...)	Add a process to the end of the ready queue.
i += 1	Move to the next process in process_list.

9. Finish the Round Robin Function

CONTINUE PART 3 DIRECTLY - PART 3C OF 4 - USE ENTER FOR NEW LINES; DO NOT RUN YET

```
p = queue.popleft()
run_time = min(quantum, remaining[p["pid"]])
start = current_time
current_time += run_time
result.append((p["pid"], start, current_time))
remaining[p["pid"]] -= run_time

while i < len(process_list) and process_list[i]["arrival"] <= current_time:
    queue.append(process_list[i])
    i += 1

if remaining[p["pid"]] > 0:
    queue.append(p)

return result
```

Term	Meaning
popleft()	Remove the first process from the FIFO queue.
run_time	Actual CPU time for this turn.
min(quantum, remaining)	Do not run longer than the quantum or the time still needed.
-=	Subtract and store the new value.
remaining > 0	Process is unfinished.
queue.append(p)	Return unfinished process to the end of the queue.

FAIRNESS RULE
Newly arrived processes are added to the queue before the unfinished running process is re-added. This preserves fair Round Robin queue order.

10. Type Program 3 - Part 4: Display and Run Both Algorithms

TYPE BELOW PART 3 - PART 4 OF 4 - USE ENTER FOR NEW LINES; DO NOT RUN YET

```
def show_result(title, result):
    print("\n" + title)
    print("Process Start End")
    for item in result:
        print(item[0], item[1], item[2])

    sequence = [item[0] for item in result]
    print("Sequence:", " -> ".join(sequence))

print("INPUT DATA")
print("PID AT BT Priority")
for p in processes:
    print(p["pid"], p["arrival"], p["burst"], p["priority"])

print("\nPriority Rule: Smaller number = Higher priority")
priority_result = priority_scheduling(processes)
show_result("NON-PREEMPTIVE PRIORITY", priority_result)

quantum = 2
print("\nRound Robin Quantum =", quantum)
rr_result = round_robin(processes, quantum)
show_result("ROUND ROBIN", rr_result)
```

NOW SAVE

The complete program is finished. Press Ctrl + S in VS Code. Ctrl + S saves the file; it does NOT run the program.

11. Run the Complete Program

STEP 7 | Open VS Code Terminal

WHERE?
In VS Code choose Terminal > New Terminal. Do not type the run command inside program3.py.

TYPE IN UBUNTU / VS CODE TERMINAL - PRESS ENTER ONCE
pwd

DO NOT TYPE - EXPECTED OUTPUT / REFERENCE ONLY
/home/your_username/OS-Third-Program

STEP 8 | Run program3.py

TYPE IN UBUNTU / VS CODE TERMINAL - PRESS ENTER ONCE
python3 program3.py

Part	Meaning
python3	Start the Python 3 interpreter.
program3.py	Execute the Python file created today.

PRESS ENTER
After typing python3 program3.py in the terminal, press Enter once. The output should appear below the command.

12. Expected Output - What Students Should See

DO NOT TYPE - EXPECTED OUTPUT / REFERENCE ONLY

INPUT DATA

PID AT BT Priority

P1 0 7 2

P2 2 4 1

P3 4 1 3

P4 5 4 2

Priority Rule: Smaller number = Higher priority

NON-PREEMPTIVE PRIORITY

Process Start End

P1 0 7

P2 7 11

P4 11 15

P3 15 16

Sequence: P1 -> P2 -> P4 -> P3

Round Robin Quantum = 2

ROUND ROBIN

Process Start End

P1 0 2

P2 2 4

P1 4 6

P3 6 7

P2 7 9

P4 9 11

P1 11 13

P4 13 15

P1 15 16

Sequence: P1 -> P2 -> P1 -> P3 -> P2 -> P4 -> P1 -> P4 -> P1

SUCCESS CHECK

If both sequences appear correctly and there is no Python error, Program 3 has executed successfully.

13. Round Robin Dry Run - Queue Movement with Quantum = 2

CPU Slice	Runs	After Slice: Ready Queue	Remaining Time
0-2	P1	P2, P1	P1=5
2-4	P2	P1, P3, P2	P2=2
4-6	P1	P3, P2, P4, P1	P1=3
6-7	P3	P2, P4, P1	P3=0 complete
7-9	P2	P4, P1	P2=0 complete
9-11	P4	P1, P4	P4=2
11-13	P1	P4, P1	P1=1
13-15	P4	P1	P4=0 complete
15-16	P1	empty	P1=0 complete

DO NOT TYPE - ROUND ROBIN EXECUTION ORDER

P1 | P2 | P1 | P3 | P2 | P4 | P1 | P4 | P1

WHY DOES A PROCESS APPEAR MORE THAN ONCE?

Round Robin is preemptive. A process that still has remaining CPU time goes back to the end of the ready queue and can receive another turn later.

14. Common Errors and Exact Solutions

Error / Problem	Likely Reason	Solution
IndentationError	Spaces under def, while, if, for do not match.	Copy indentation exactly. In Python, indentation is part of syntax.
NameError: deque is not defined	First import line is missing.	Add: from collections import deque
SyntaxError near lambda / brackets	Missing comma, bracket, quote, or colon.	Compare the current line with the guide; check (), [], {}, commas and colons.
Quantum must be greater than 0	quantum is 0 or negative.	Set quantum = 2 for the standard practical.
python3: can't open file	Terminal is not inside OS-Third-Program.	Run cd ~/OS-Third-Program and then python3 program3.py.
Attribute / key error	Dictionary key typed incorrectly.	Use exact keys: pid, arrival, burst, priority.
Wrong Priority sequence	Priority convention or ready-process rule changed.	Use smaller number = higher priority and choose only arrived unfinished processes.
Wrong Round Robin sequence	Queue order or re-adding rule is wrong.	Add newly arrived processes before re-adding the unfinished running process.

FIRST DEBUGGING RULE

Read the LAST line of the Python error first. It normally tells you the error type and often the line number. Do not rewrite the whole program immediately.

15. What to Explain on the Board + Oral Questions

BOARD NOTES

WRITE / EXPLAIN ON BOARD - STUDENTS DO NOT TYPE THIS AS CODE

PROGRAM 3

Priority + Round Robin

AT = Arrival Time

BT = Burst Time

Priority:

Smaller number = Higher priority

Non-preemptive

Round Robin:

Quantum = 2

Preemptive

FIFO Ready Queue

Remaining Time

= CPU time still needed

ORAL QUESTIONS

1. What is Priority Scheduling?

Select the ready process with the best priority.

2. Which priority is higher: 1 or 3?

1, because smaller number is higher here.

3. Is our Priority algorithm preemptive?

No, it is non-preemptive.

4. What is a time quantum?

Maximum CPU time in one Round Robin turn.

5. Why can P1 appear many times in RR?

It needs more CPU time than one quantum.

6. What is the ready queue?

FIFO queue of processes waiting for CPU.

16. End-of-Class Checklist and Quick Command Card

STUDENT CHECKLIST

- ☐ OS-Third-Program folder created.
- ☐ program3.py created in VS Code.
- ☐ All four code parts typed in the same file.
- ☐ Ctrl + S used to save.
- ☐ python3 program3.py runs without error.
- ☐ Priority rule explained correctly.
- ☐ Priority sequence = P1 -> P2 -> P4 -> P3.
- ☐ Quantum = 2.
- ☐ RR sequence has repeated process turns.
- ☐ Student can explain remaining time and ready queue.

QUICK COMMAND CARD

TYPE THESE IN TERMINAL - ONE COMMAND AT A TIME

```
cd ~  
ls  
mkdir -p ~/OS-Third-Program  
cd ~/OS-Third-Program  
pwd  
code .
```

```
# After program3.py is written and saved:  
python3 program3.py
```

OPEN AN OLDER PROGRAM LATER

```
# Program 1  
cd ~/OS-First-Program  
code .  
  
# Program 2  
cd ~/OS-Second-Program  
code .  
  
# Program 3  
cd ~/OS-Third-Program  
code .
```